Functional programming in Java, Part 2: Lambda reuse, lexical scoping and closures, and reduce()

Reusing lambda expressions

Using lexical scoping and closures

Picking a single element

Reducing a collection to a single value

Joining elements

Conclusion

Dig deeper

CODING

Functional programming in Java, Part 2: Lambda reuse, lexical scoping and closures, and reduce()

Learn how to use lambda expressions to greatly reduce code clutter.

by Venkat Subramaniam

June 11, 2021

[This article was adapted from *Functional Programming in Java: Harnessing the Power of Java 8 Lambda Expressions* with kind permission from the publisher, The Pragmatic Bookshelf. —*Ed*.]

Lambdas create more-expressive and concise code with less mutability and fewer errors. In the first article in this two-part series, I demonstrated how lambda expressions harness the power of the functional style of programming in Java. In this final part, I explore this further and consider a cautionary warning. (I suggest you read Functional programming in Java, Part 1: Lists, lambdas, and method references, if you haven't already.)

As you'll see, lambda expressions are deceivingly concise, and it's easy to carelessly duplicate them in code. Duplicate code leads to poor-quality code that's hard to maintain; if you needed to make a change, you'd have to find and touch the relevant code in several places.

Avoiding duplication can also help improve performance. By keeping the code related to a piece of knowledge concentrated in one place, you can easily study its performance profile and make changes in one place to get better performance.

Reusing lambda expressions

I'll demonstrate how easy it is to fall into the duplication trap when using lambda expressions and suggest ways to avoid that trap. Suppose you have a few collections of names: `friends`, `editors`, and `comrades`.

```
final List<String> friends =
   Arrays.asList("Brian", "Nate", "Neal", "Ra
final List<String> editors =
   Arrays.asList("Brian", "Jackie", "John", "
final List<String> comrades =
   Arrays.asList("Kate", "Ken", "Nick", "Paul
```

The goal is to filter out names that start with a certain letter. First, here is a naive approach to this task using the `filter()` method.

```
final long countFriendsStartN =
   friends.stream()
      .filter(name -> name.startsWith("N"))
      .count();
final long countEditorsStartN =
   editors.stream()
      .filter(name -> name.startsWith("N"))
      .count();
final long countFriendsStartN =
   friends.stream()
      .filter(name -> name.startsWith("N"))
      .count();
```

The lambda expressions made the code concise, but they quietly led to duplicate code. In the previous example, one change to the lambda expression requires changes in more than one place—that's a no-no. Fortunately, you can assign lambda expressions to variables and reuse them, as you would with objects.

The `filter()` method, the receiver of the lambda expression in the previous example, takes a reference to a `java.util.function.Predicate` functional interface. Here, the Java compiler works its magic to synthesize an implementation of the `Predicate`'s `test()` method from the given lambda expression. Rather than asking Java to synthesize the method at the argument-definition location, you can be more explicit. In this example, it's possible to store the lambda expression in an explicit reference of type `Predicate` and then pass it to the function. This is an easy way to remove the duplication.

Here's how to refactor the previous code to make it adhere to the DRY (Don't Repeat Yourself) best practice.

```
final Predicate<String> startsWithN =
   name -> name.startsWith("N");
final long countFriendsStartN =
   friends.stream()
      .filter(startsWithN)
      .count();
final long countEditorsStartN =
   editors.stream()
      .filter(startsWithN)
```

```
        .count();
final long countComradesStartN =
   comrades.stream()
      .filter(startsWithN)
      .count();
```

Rather than duplicate the lambda expression several times, you created it once and stored it in a reference named `startsWithN` of type `Predicate`. In the three calls to the `filter()` method, the Java compiler happily took the lambda expression stored in the variable under the guise of the `Predicate` instance.

The new variable gently removed the duplication that sneaked in. Unfortunately, the duplication is about to sneak back in with a vengeance, as you'll see next, and you need something a bit more powerful to thwart it.

Using lexical scoping and closures

Some developers believe that using lambda expressions might introduce duplication and lower code quality. Contrary to that belief, even when the code gets more complicated, you still don't need to compromise code quality to enjoy the conciseness given by lambda expressions.

You managed to reuse the lambda expression in the previous example; however, duplication will sneak in quickly when you bring in a second letter to match. Let's explore the problem further and then solve it using lexical scoping and closures.

Duplication in lambda expressions. Let's pick the names that start with *N* or *B* from the `friends` collection of names. Continuing with the previous example, you might be tempted to write something like the following:

```
final Predicate<String> startsWithN =
   name -> name.startsWith("N");
final Predicate<String> startsWithB =
   name -> name.startsWith("B");
final long countFriendsStartN =
   friends.stream()
      .filter(startsWithN)
      .count();
final long countFriendsStartB =
   friends.stream()
      .filter(startsWithB)
.count();
```

The first predicate tests whether the name starts with an *N* and the second tests for a *B*. You pass these two instances to the two calls to the `filter()` method, respectively. That seems reasonable, but the two predicates are duplicates, with only the letter they use being different. The goal is to eliminate this duplication.

Removing duplication using lexical scoping. As a first option, you could extract the letter as a parameter to a function and pass the function as an argument to the `filter()` method. That's a reasonable idea, but `filter()` will not accept any arbitrary function; it insists on receiving a function that accepts one parameter representing the context element in the collection, and it returns a `Boolean` result. It's expecting a `Predicate`.

For comparison purposes, you need a variable that will cache the letter for later use and hold onto it until the parameter, `name`, in this example, is received. Here's how to create a function for that.

```
public static Predicate<String>
   checkIfStartsWith(final String letter) {
   return name -> name.startsWith(letter);
}
```

This defined `checkIfStartsWith()` as a `static` function that takes a `letter` of type `String` as a parameter. It then returns a `Predicate` that can be passed to the `filter()` method for later evaluation; `checkIfStartsWith()` returns a function as a result.

The `Predicate` that `checkIfStartsWith()` returned is different from the lambda expressions you've seen so far. In `return name -> name.startsWith(letter)`, it is clear what `name` is: It's the parameter passed to this lambda expression.

But what's the variable `letter` bound to? Because that's not in the scope of this anonymous function, Java reaches over to the scope of the definition of this lambda expression and finds the variable `letter` in that scope. This is called *lexical scoping*. Lexical scoping is a powerful technique that lets you cache values provided in one context for use later in another context. Since this lambda expression closes over the scope of its definition, it's also referred to as a *closure*.

It's worth noting here that there are a few restrictions to lexical scoping. For one thing, from within a lambda expression, you can access only local variables that are `final` or effectively `final` in the enclosing scope. A lambda expression may be invoked right away, or it may be invoked lazily or from multiple threads.

To avoid race conditions, the local variables you access in the enclosing scope are not allowed to change once they are initialized. Any attempt to change them will result in a compilation error. Variables marked `final` directly fit this bill, but Java does not insist that you mark them as such. Instead, Java looks for the following two things:

- The accessed variables must be initialized within the enclosing methods before the lambda expression is defined.
- The values of these variables don't change anywhere else —that is, they're effectively `final` even if they are not marked as such.

When you use lambda expressions that capture local state, you should also be aware that stateless lambda expressions are runtime constants, but those that capture local state have an additional evaluation cost.

With these restrictions in mind, let's see how to use the lambda expression returned by `checkIfStartsWith()` in the calls to the `filter()` method.

```
final long countFriendsStartN =
   friends.stream()
      .filter(checkIfStartsWith("N"))
      .count();
final long countFriendsStartB =
   friends.stream()
      .filter(checkIfStartsWith("B"))
      .count();
```

In the calls to the `filter()` method, you first invoke the `checkIfStartsWith()` method, passing in a desired letter. This call immediately returns a lambda expression that is then passed on to the `filter()` method.

By creating a higher-order function, `checkIfStartsWith()`, in this example, and by using lexical scoping, you managed to remove the duplication in code. You did not have to repeat the comparison to check whether the name starts with different letters.

Refactoring to narrow the scope. In the preceding example, you used a `static` method, but you don't want to pollute the class with `static` methods to cache each variable in the future. It would be nice to narrow the function's scope to where it's needed. You can accomplish that by using a `Function` interface.

```
final Function<String, Predicate<String>>
   startsWithLetter = (String letter) -> {
      Predicate<String> checkStarts =
         (String name) -> name.startsWith(let
   return checkStarts;
};
```

This lambda expression replaces the `static` method `checkIfStartsWith()` and can appear within a function, right before it's needed. The `checkIfStartsWith` variable refers to a `Function` that takes in a `String` and returns a `Predicate`.

This version is verbose compared to the `static` method you saw earlier, but you'll refactor it soon to make it concise. For all practical purposes, this function is equivalent to the `static` method; it takes a `String` and returns a `Predicate`. Instead of explicitly creating the instance of the `Predicate` and returning it, you can replace it with a lambda expression.

```
final Function<String, Predicate<String>>
   startsWithLetter =
      letter -> name -> name.startsWith(lette
```

You've come full circle with higher-order functions in this section. The examples illustrate how to pass functions to functions, create functions within functions, and return functions from within functions. They also demonstrate the conciseness and reusability that lambda expressions facilitate.

You made good use of both `Function` and `Predicate` in this section, but let's discuss how they're different.

- A `Predicate<T>` takes in one parameter of type `T` and returns a `Boolean` result to indicate a decision for whatever check it represents. You can use it anytime you want to make a go or no-go decision for a candidate you pass to the `Predicate`. Methods such as `filter()` that evaluate candidate elements take in a `Predicate` as their parameter.
- A `Function <T, R>` represents a function that takes a parameter of type `T` and returns a result of type `R`. This is more general than a `Predicate` that always returns a boolean. You can employ a `Function` anywhere you want to transform an input to another value, so it's quite logical that the `map()` method uses `Function` as its parameter.

Selecting all the matching elements from a collection was easy. Next, I'll show you how to pick a single element out of a collection.

Picking a single element

It's reasonable to expect that picking one element from a collection would be simpler than picking multiple elements, but that's not always true. I'll start by exploring the complexity introduced by the habitual approach and then bring in lambda expressions to reduce that complexity.

Let's create a method that looks for an element that starts with a given letter and prints it.

```
public static void pickName(
   final List<String> names,
   final String startingLetter) {
      String foundName = null;
      for(String name : names) {
         if(name.startsWith(startingLetter))
```

```
                foundName = name;
                break;
            }
        }
    System.out.print(
        String.format(
            "A name starting with %s: ",
            startingLetter));

    if(foundName != null) {
        System.out.println(foundName);
    } else {
        System.out.println("No name found");
    }
}
```

This method's smell can easily compete with passing garbage trucks. You first created a `foundName` variable and initialized it to `null`—that's the source of the first bad smell. This will force a `null` check, and if you forget to deal with that check, the result could be a `NullPointerException` or some other unpleasant response.

You then used an external iterator to loop through the elements but had to break out of the loop if you found an element—here are other sources of rancid smells: primitive obsession, imperative style, and mutability. Once out of the loop, you had to check the response and print the appropriate result. That's quite a bit of code for a simple task.

Let's rethink the problem. You simply want to pick the first matching element and safely deal with the absence of such an element. Let's rewrite the `pickName()` method using lambda expressions.

```
public static void pickName(
    final List<String> names,
    final String startingLetter) {
        final Optional<String> foundName =
        names.stream()
            .filter(name -> name.startsWith(star
            .findFirst();
        System.out.println(
            String.format( "A name starting with
                startingLetter, foundName.orElse(
}
```

Some powerful features in the JDK library came together to help achieve this conciseness. First, the `filter()` method grabbed all the elements matching the desired pattern. Then the `findFirst()` method of the `Stream` class helped pick the first value from that collection. This method returns a special `Optional` object, which is the state-appointed `null` deodorizer in Java.

The `Optional` class is useful whenever the result may be absent. It protects you from getting a `NullPointerException`

by accident and makes it quite explicit to the reader that “no result found” is a possible outcome.

You can inquire whether an object is present by using the `isPresent()` method, and you can obtain the current value using its `get()` method. Alternatively, you could suggest a substitute value for the missing instance, using the method (with the vaguely threatening name) `orElse()`, as in the previous code. Let’s exercise the `pickName()` function with the sample `friends` collection you’ve used in the examples so far.

```
pickName(friends, "N");
pickName(friends, "Z");
```

The code picks out the first matching element, if one is found, and prints an appropriate message otherwise.

```
A name starting with N: Nate
A name starting with Z: No name found
```

The combination of the `findFirst()` method and the `Optional` class reduced the code and its smell quite a bit. You’re not limited to the preceding options when working with `Optional`, however. For example, rather than providing an alternate value for the absent instance, you can ask `Optional` to run a block of code or a lambda expression only if a value is present, as follows:

```
foundName.ifPresent( name ->
   System.out.println("Hello " + name));
```

Compared to using the imperative version to pick the first matching name, the nice, flowing functional style looks better. But are you doing more work in the fluent version than you did in the imperative version? The answer is no—these methods have the smarts to perform only as much work as necessary.

The search for the first matching element demonstrated a few more neat capabilities in the JDK. Next, I’ll show you how lambda expressions help compute a single result from a collection.

Reducing a collection to a single value

I’ve gone over quite a few techniques to manipulate collections so far: picking matching elements, selecting a particular element, and transforming a collection. All these operations have one thing in common: They all worked independently on individual elements in the collection. None required comparing elements

against each other or carrying over computations from one element to the next.

In this section, I will compare elements and carry over a computational state across a collection. This will begin with some basic operations and then build up to something a bit more sophisticated. As the first example, you are going to read over the values in the `friends` collection of names and determine the total number of characters.

```
System.out.println(
   "Total number of characters in all names:
   friends.stream()
      .mapToInt(name -> name.length())
      .sum());
```

To find the total of the characters, you need the length of each name, which can be found using the `mapToInt()` method. Once you transform the names to their lengths, the final step is to add them, which is performed using the built-in `sum()` method. Here's the output for this operation.

```
Total number of characters in all names: 26
```

This code leveraged the `mapToInt()` method, a variation of the `map()` operation (variations such as `mapToInt()` and `mapToDouble()` create type-specialized streams such as `IntStream` and `DoubleStream`), and then *reduced* the resulting length to the `sum` value.

Instead of using the `sum()` method, of course, the code could use a variety of methods, such as `max()` to find the longest length, `min()` to find the shortest length, `average()` to find the average lengths, and so on.

The hidden charm in the preceding example is the increasingly popular MapReduce pattern, with the `map()` method being the spread operation and the `sum()` method being the special case of the more general reduce operation. In fact, the implementation of the `sum()` method in the JDK uses a `reduce()` method. Here's a look at the more general form of the reduce operation.

As an example, the task is to read through the given collection of names and display the longest one. If there is more than one name with the same longest length, you'll display the first one you find. One way you could do that is to figure out the longest length and then pick the first element of that length. But that would require going through the list twice, which is not efficient. This is where a `reduce()` method comes into play.

You can use the `reduce()` method to compare two elements against each other and pass along the result for further

comparison with the remaining elements in the collection. Much like the other higher-order functions on collections you've seen so far, the `reduce()` method iterates over the collection. In addition, `reduce()` carries forward the result of the computation that the lambda expression returns. An example will help clarify this.

```
final Optional<String> aLongName =
   friends.stream()
      .reduce((name1, name2) ->
         name1.length() >= name2.length() ? n
   aLongName.ifPresent(name ->
      System.out.println(
         String.format("A longest name: %s",
```

The lambda expression you are passing to the `reduce()` method takes two parameters, `name1` and `name2`, and returns one of them based on the length. The `reduce()` method, of course, has no clue about the application logic's specific intent. That concern is separated from this method into the lambda expression that you pass to it—this is a lightweight application of the Strategy pattern.

This lambda expression conforms to the interface of an `apply()` method of a JDK functional interface named `BinaryOperator`. This is the type of the parameter the `reduce()` method receives. Let's see if the `reduce()` method picks the first of the two longest names from the `friends` list.

```
A longest name: Brian
```

As the `reduce()` method iterated through the collection, it called the lambda expression first, with the first two elements in the list. The result from the lambda expression is used for the subsequent call. In the second call, `name1` is bound to the result from the previous call to the lambda expression, and `name2` is bound to the third element in the collection. The calls to the lambda expression continue for the rest of the elements in the collection. The result from the final call is returned as the result of the `reduce()` method call.

The result of the `reduce()` method is an `Optional` because the list on which `reduce()` is called might be empty, and in that case, there would be no longest name. If the list had only one element, `reduce()` would return that element and the lambda expression would not be invoked.

From the example, you can infer that the `reduce()` method's result is at most one element from the collection. If you want to set a default or a base value, you can pass that value as an extra parameter to an overloaded variation of the `reduce()` method. For example, if the default shortest name you want to

pick is “Steve,” you can pass that to the `reduce()` method, as follows:

```
final String steveOrLonger =
   friends.stream()
      .reduce("Steve", (name1, name2) ->
         name1.length() >= name2.length() ? n
```

If any name is longer than the given base, it is picked up; otherwise, the function returns the base value, which is “Steve” in this example. This version of `reduce()` does not return an `Optional` because if the collection is empty, the default of “Steve” will be returned; there is no concern about an absent or nonexistent value.

Before I wrap up, let’s visit a fundamental, yet seemingly difficult, operation on collections: joining elements.

Joining elements

You’ve explored how to select elements, iterate, and transform collections. Yet in a trivial operation—concatenating a collection—you could lose all the gains you made from concise and elegant code if not for the `join()` function, introduced in JDK 8. This simple method is so useful that it’s poised to become one of the most used functions in the JDK. Let’s see how to use it to print the values in a comma-separated list.

Going back to the `friends` list: What does it take to print the list of names, separated by commas, without using `join()`? You have to iterate through the list and print each element. Because the enhanced Java 5 `for` construct is better than the archaic `for` loop, let’s start with that.

```
for(String name : friends) {
   System.out.print(name + ", ");
}
System.out.println();
```

That simple code yields the following:

```
Brian, Nate, Neal, Raju, Sara, Scott,
```

Uh oh! There’s a stinking comma at the end (can that be blamed on Scott?). How do you tell Java not to place a comma there? Unfortunately, the loop will run its course and there’s no easy way to tell the last element apart from the rest. To fix this, you can fall back on the habitual loop.

```
for(int i = 0; i < friends.size() - 1; i++) {
   System.out.print(friends.get(i) + ", ");
}
if(friends.size() > 0)
   System.out.println(friends.get(friends.siz
```

Let's see if the output of this version is decent.

```
Brian, Nate, Neal, Raju, Sara, Scott
```

The output looks good, but the code to produce the output does not. Beam me up, modern Java.

Good news: The `StringJoiner` class added in Java 8 cleans up that mess. The `String` class has an added convenience method, `join()`, to turn that smelly code into a simple one-liner.

```
System.out.println(String.join(", ", friends)
```

The following verifies that the output is as charming as the code that produced it:

```
Brian, Nate, Neal, Raju, Sara, Scott
```

Under the hood, the `String.join()` method calls upon the `StringJoiner` to concatenate the values in the second argument, a `varargs`, into a larger string separated by the first argument. You're not limited to concatenating only with a comma using this feature. You could, for example, take a bunch of paths and concatenate them to form a classpath easily, thanks to the new methods and classes.

You can also transform the elements before joining them. You already know how to transform elements using the `map()` method. You can also be selective about which elements you want to keep by using methods such as `filter()`. The final step of joining the elements, separated by commas or something else, is simply a `reduce` operation.

You could use the `reduce()` method to concatenate elements into a string, but that would require more effort than is necessary. The JDK has a convenience method named `collect()`, which is another form of `reduce` that can collect values into a target destination.

The `collect()` method does the reduction but delegates the actual implementation or target to a collector. You could drop the transformed elements into an `ArrayList`, for instance. Or, to

continue with the current example, you could collect the transformed elements into a string concatenated with commas.

```
System.out.println(
   friends.stream()
      .map(String::toUpperCase)
      .collect(joining(", ")));
```

This code invoked the `collect()` method on the transformed list and provided it a collector returned by the `joining()` method, which is a `static` method on a `Collectors` utility class. A collector acts as a sink object to receive elements passed by the `collect()` method and stores them in a desired format, such as `ArrayList` or `String`.

Here are the names, now in uppercase and neatly separated by commas.

```
BRIAN, NATE, NEAL, RAJU, SARA, SCOTT
```

The `StringJoiner` gives a lot more control over the format of concatenation; you can specify a prefix, a suffix, and infix character sequences, if you desire.

Conclusion

As you've seen in this two-part series, lambda expressions and other classes and methods make programming in Java so much easier and more fun, too.

Collections are common in programming and, thanks to lambda expressions, using them is now much easier and simpler in Java. You can trade the long-winded old methods for elegant, concise code to perform common operations on collections. Internal iterators make it convenient to traverse collections, transform collections without enduring mutability, and select elements from collections without much effort. Using these functions means less code to write. That can lead to more maintainable code, more code that does useful domain- or application-related logic, and less code to handle the basics of coding.

Dig deeper

- Behind the scenes: How do lambda expressions really work in Java?
- Venkat Subramaniam: Java is changing in a responsible manner
- The Java tutorial for lambda expressions
- The Java tutorial for method references
- Java 8: Lambdas, Part 1

- Java 8: Lambdas, Part 2

Venkat Subramaniam

Venkat Subramaniam is an award-winning author, founder of Agile Developer, Inc., and an adjunct faculty at the University of Houston. Subramaniam is author and co-author of multiple books, including the 2007 Jolt Productivity award winning book *Practices of an Agile Developer*. Subramaniam has trained and mentored thousands of software developers in the US, Canada, Europe, and Asia, and is a regularly-invited speaker at several international conferences. His latest book is *Pragmatic Scala: Create Expressive, Concise, and Scalable Applications*.

Share this Page

Contact
US Sales: +1.800.633.0738
Global Contacts
Support Directory
Subscribe to Emails

About Us
Careers
Communities
Company Information
Social Responsibility Emails

Downloads and Trials
Java for Developers
Java Runtime Download
Software Downloads
Try Oracle Cloud

News and Events
Acquisitions
Blogs
Events
Newsroom

ORACLE | Integrated Cloud
Applications & Platform Services

© Oracle | Site Map | Terms of Use & Privacy | Cookie Preferences | Ad Choices